

INF-122 – Funksjonell programmering, H-2026

forelesninger: man.8:15-10, tirs.14:15-16 – Michal.Walicki@uib.no

administrative spørsmål (eksamen, mitt-uib): – Jenny.Thunes@uib.no

Grupper er kun for å diskutere løsningsforslag dere har laget:

- ▶ du må selv forsøke å løse oppgaver på forhånd
- ▶ ingen løsningsforslag blir serevert
- ▶ meld deg på én tutorial-stil/diskusjonsgruppe
 - 1) torsdag 10:15, fredag 10:15 el. fredag 12:15 (på mitt-uib)
 - 2) eller epost til Johannes.Skivdal@student.uib.no eller James.Hobson@uib.no – for å avtale tidspunkt for mindre/kortere grupper (2-5pers/30min; James' gruppe blir på engelsk)
- ▶ Hverken gruppepåmelding eller -deltakelse er obligatorisk, men ...

NB! AI kan løse problemer på dette nivå: vil du lære så bruker du den ikke.

For å kunne gå opp til eksamen, må man få godkjent:

- ▶ én større obligatorisk oppgave (oktober/november)
- ▶ ingen delkarakter på denne, kun godkjent/ikke-godkjent;
- ▶ det blir neppe tid til å rette på en ikke-godkjent innlevering!

Foreløpig pensum og **tentativ** framdriftsplan

(A) “**Programming in Haskell**”, G.Hutton, 2016 (kap.1-8,10,15)

(B) **Alle forelesningsnotater.**

(C) Alle oppgaver fra boken(s kapitler som gjennomgås) – antas løst.

uke 34: 1–2 (15) (kapittel)

35–36: 3–4

37–40: 5–8

41–42: 10

42–43: enkel parsing (**kun forelesningsnotater!**)

43–44: typeinferens (**kun forelesningsnotater!**)

44–46: Oblig (frist antakelig rund 10 november)

47: resultater og gjennomgang av Oblig

48: spørsmål og svar

49: 2.12 – Eksamen

Én forelesning om funksjonell programmering i næringslivet ??

Funksjonell programmering – prinsipper:

- ▶ Basert på **evaluering** av uttrykk

dvs., resultat avhenger kun av verdier til deluttrykk, ikke av tilstand

- ingen bieffekter (side-effects); ingen tilordning til variabler
- implisitt minne-forvaltning (inkl. 'garbage collection')

- ▶ Funksjoner er første-klasses objekter

- ▶ Se http://en.wikipedia.org/wiki/Functional_programming

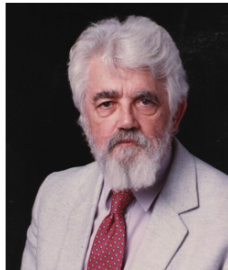
Kort historie

1930s:



Alonzo Church
lambda kalkyle,
– en enkel men
kraftig teori
av **funksjoner**
(Turing-komplett).

1950s:



John McCarthy *Lisp*
(1958): utypet

- ▶ det første FP språket
- ▶ påvirket av lambda kalkyle
- ▶ med variabel-tilordning.

1960s, LISP dialekter:

- ▶ Common Lisp
- ▶ Scheme

1970s, *ML*: typet



Robin Milner

- ▶ CAML, OCAML
- ▶ SMLNJ

1990: Haskell

Hvorfor FP?

- ▶ Meget konsis kode (men, selvfølgelig, Turing-komplett), f.eks.:

Java:

```
int total = 0;
for (int i = 1; i <= 50; i++)
    total = total + i;
```

Haskell:

```
sum [1..50]
```

- ▶ Øker betraktelig korrekthet – en oversiktlig formell semantikk
(likhetsresonnering om programmer)
- ▶ Kilde til flere konsepter – viktige også i imperativ programmering:
 - ▶ rekursjon (datastrukturer og algoritmer)
 - ▶ typer (eksplisitt og implisitt)
 - ▶ polymorfi (gjenbruk av algoritmer)
 - ▶ høyere-orden (funksjoner inn og ut)
- ▶ Viktige implementasjonsteknikker:
 - ▶ type-sjekking, type-avledning
 - ▶ lokale og globale variabler (stabelmodellen)
 - ▶ 'garbage collection'
- ▶ Brukes i praksis, f.eks. banker (JPMorgan, Klarna), Finn.no, Frende, Heathrow-London github.com/erkmos/haskell-companies

Imperativt

```
swap-a(x,y: int) {  
  var z := y;  
  y := x;  
  x := z; }
```

uten ekstra minne

```
void swap(x,y: int) {  
  x := x + y;  
  y := x - y;  
  x := x - y; } – kall ved navn
```

z := 3;

swap(z,z)

z = 0

tilordning under aliasing (tilstandshåndtering m.sideeffekter)
er vanskelig

Funksjonelt

```
swap(x,y) = (y,x)    polymorf!  
swap(2,1)  $\rightsquigarrow$  (1,2)  
swap(2,True)  $\rightsquigarrow$  (True,2)  
...
```

abstraherer fra tilstand

dvs.,

krever ikke manipulering
av lager (“variabler”)

Kompilering vs. tolkning

```
void inc(x:int)
  if (x*3 + 3 > 0)
    y := x*3 + 4;
  else y := x*3 + 3;
```

```
inc(x) =
  if (x*3 + 3 > 0)
    x*3 + 4
  else x*3 + 3
```

```
inc(2) =
  if (2*3
    2*
  else 2*
```

Kompilering ↓

inc: (maskinkode)

```
1. LD(r1, x2)
2. LD(r2, r1 * 3)
3. LD(r3, r2 + 3)
4. IF(r3 > 0, GOTO 7)
5. LD(y, r1)
6. GOTO 8
7. LD(y, r3 + 1)
8. HALT
```

inc(2) ↑ y = 10

resultat: tilstandsending

inc(2)

```
ev( if True a else b ) = ev(a)
ev( if False a else b ) = ev(b)
ev( if X a else b ) = ev(if ev(X) a else b)
ev( x "+" y ) = ev(x) + ev(y)
ev( x "*" y ) = ev(x) * ev(y)
...
```

~ 10

resultat: en verdi

Viktige ressurser

- ▶ <http://www.haskell.org> – Haskell hjemmeside
installer GHCi (Haskell Platform) – ikke kun GHC
<https://www.haskell.org/ghcup/> (evt., Hugs)
- ▶ for å installere ekstra pakker, kjør fra terminal kommando:
> `cabal install -lib QuickCheck`
- ▶ <http://www.cs.nott.ac.uk/~pszgmh/pih.html>
– bokens web-side (errata, notater, video, mm.)

bok: “Learn You A Haskell for Great Good”
– <http://learnyouahaskell.com>

bok: “Real World Haskell”
– <http://www.realworldhaskell.org>

- ▶ mange forelesninger på Youtube, som Computerphile

1. Installer Haskell (ghci) fra www.haskell.org på din maskin.
2. Programmer skrives i egen fil (teksteditor) med utvidelsen `.hs`
3. `> ghci filnavn.hs`
– starter ghci (fra terminal) og laster koden fra filen `filnavn.hs`
4. og filen lastes (fra ghci/Hugs) med
`:load filnavn.hs`, eller
`:l filnavn`
5. filen blir **type-sjekket** og, hvis den er korrekt,
kan alle funksjoner kalles interaktivt fra GHCi (ingen Main)
6. `:reload` – laste om programmet etter redigering
`:r`
7. `:quit` – avslutt GHCi/Hugs
`:q`

Prog $\longrightarrow$ funksjonsnavn argumentliste = uttrykk { Prog }

Skrives i egen fil (teksteditor) og filen lastes (fra ghci/Hugs) med
Interaktivt, kan man bruke **let** for

:load filnavn.hs eller å definere funksjoner og variabler
“lokalt” i

:l filnavn denne sesjonen av ghci

- ▶ **let** inc n = n + 1 – prøv “inc 13”
- ▶ **let** two = 2 – prøv “inc two”
- ▶ **let** fst (x,y) = x – Obs: hva betyr parentesene?
- ▶ **let** fst1 x y = x
- ▶ **let** plus x y = x + y
- ▶ **let** suc = plus 1 – prøv “suc two”
..... – deretter “suc suc two” og “suc (suc two)”

En slik **let** er ikke nødvendig i nyere versjoner av GHCi.

Eksempler på lokale bindinger: 'let...in...'

- a. **let** x = 2 **in** x*x + 1
- b. **let** x = 2 **in** **let** y = -1 **in** x*y + 1
 let x = 2 ; y = -1 **in** x*y + 1
 let x = 2 **in** x*y + 1
 where y = -1
- c. **let** x = 2 **in** (**let** x = -1 **in** x) -1
 let x = 2 **in** x + (**let** x = -1 **in** x) 1
 let x = 2 ; x = -1 **in** x - feil
 versus imperativt: x := 2 ; x := -1
- d. **let** x = 2 ; z = -1 **in** x*y + z + 1 - feil
 f 2 - feil
- e. **let** a=4; f x = x+a **in** **let** a=0 **in** f a 4
- f. **let** x = 2 ; y =x*x **in** y*y 16
- g. **let** y =x*x ; x = 2 **in** y*y 16 - **lat !**

Nyttige ting (Haskell)

- funksjons- og variabelnavn starter med **småbokstav**
- Navn på data-konstruktorer starter med **storbokstav** (True,False...)

Paranteser!!!

$f\ a\ b \neq f(a,b)$

anta $a :: A$ og $b :: B$

$f :: A \rightarrow (B \rightarrow X)$

$f :: (A, B) \rightarrow X$

kan evaluere $f\ a :: B \rightarrow X$

$f(a,)$ - kan ikke evalueres

`-- 1` er feil og `-- 1` er kommentar

`-( -1)` er riktig

$f\ a + b$

skrives i matte

$f(a) + b$

– funksjonsapplikasjon binder sterkest

$f\ a\ b + c*d$

skrives i matte

$f(a,b) + c*d$

$f\ a\ (g\ b)$

skrives i matte

$f(a, g(b))$

$f\ a\ g\ b$

skrives i matte

$f(a, g, b)$

$(f\ a\ g\ b)$

skrives i matte

$f(a, g, b)$

binær $f\ x\ y = \dots$, kan kalles som infiks: $a\ 'f'\ b$ (med backquotes).

`--` kommentar i én linje

`{-` start av en kommentar som avsluttes med `-}`

Nyttige ting og noen konvensjoner...

1. Definisjoner MÅ starte i **samme kolonne** – indentering teller!

f = ...	f = ...
g = ...	g = ...
h = ...	h = ...
OK	FEIL

2. Indentering KAN brukes for **gruppering**:

a = b + c	a = b + c
where	where
b = 2	{ b = 2 ; c = 3 }
c = 3	
d = a * 2	d = a * 2
implisitt	eksplisitt med { }, ikke ()

3. Listeargumenter har, som en konvensjon, `_s` på slutten:

`add x xs :: a -> [a] -> [a]`

ingen løkker...

$$\text{fact } n = \prod_{i=1}^{i=n} i$$

```
fact n =  
  res := 1  
  for (i:=1, i++, n)  
    res := res * i  
  return res
```

... kun rekursjon

- ▶ `fact n = if n <= 0 then 1 else n * fact (n-1)`
- ▶ `fact1 n = product [1..n]`
- ▶ `fact2 0 = 1`
`fact2 n = n * fact2(n-1)`
- ▶ `fib n = if n <= 1 then 1`
`else fib (n-1) + fib (n-2)`
- ▶ `fib1 0 = 1`
`fib1 1 = 1`
`fib1 n = fib1 (n-1) + fib1 (n-2)`

Rekursjon: lister og mønstre

tom liste [], eller $x:xs$ $[1,2,3] = 1:[2,3] = 1:2:[3] = 1:2:3:[]$

- ▶ $sm1\ xs = \text{if } [] == xs \text{ (null } xs) \text{ then } 0 \text{ else head } xs + sm1\ tail\ xs$
 $\text{else (head } xs) + (sm1\ (tail\ xs))$
- ▶ $sm2\ [] = 0$
 $sm2\ (x:xs) = x + sm2\ xs$
- ▶ $append\ []\ ys = ys$
 $append\ (x:xs)\ ys = x : append\ xs\ ys$ -- innebygget: $xs ++ ys$
- ▶ $rev\ [] = []$
 $rev\ (x:xs) = rev(xs) : x$ -- innebygget som *reverse*
- ▶ $ins\ [] = []$ -- sett inn listens første elementet ved første
 $ins\ [x] = [x]$ -- posisjonen med større påfølgende element
 $ins\ (x:y:xs) = \text{if } x \leq y \text{ then } x:y:xs \text{ else } y:ins(x:xs)$
- ▶ $inss\ [] = []$ -- "innstikk" sortering
 $inss\ (x:xs) = ins(x:xs)inss(xs)$

Noen sentrale listefunksjoner

typen 'liste av a-elementer' skrives som `[a]`

`head :: [a] -> a` `last :: [a] -> a`

`tail :: [a] -> [a]` `init :: [a] -> [a]`

`++ :: [a] -> [a] -> [a]`

`length :: [a] -> Int`

`reverse :: [a] -> [a]`

`take :: Int -> [a] -> [a]`

`take 2 [1,2,3,4,5] = [1,2]`

`drop :: Int -> [a] -> [a]`

`drop 2 [1,2,3,4,5] = [3,4,5]`

`!! :: [a] -> Int -> a`

`[5,6,7,8] !! 1 = 6`

`[5,6,7,8] !! 0 = 5`

`lis !! length lis` *** Exception. Prelude(!): Index too large

`sum :: Num a => [a] -> a`

`sum [1,2,3,4] = 10`

`product :: Num a => [a] -> a`

`product [1,2,3,4] = 24`

Eksempler...

- ▶ `like xs ys = ?` `like [] [] = True`
`like [] xs = False`
`like xs [] = False`
`like (x:xs) (y:ys) = (x == y) && (like xs ys)`

bedre: gitt lister `xs` og `ys`, boolsk likhetstest er: `xs == ys`

- ▶ `last [x] = x`
`last (x:y:ys) = last (y:ys)`

eller `last xs = head (reverse xs)`

eller `last xs = xs !! (length xs - 1)`

- ▶ `init [x] = []` - - fjern siste elementet
`init (x:xs) = x : (init xs)`

eller `init xs = reverse (tail (reverse xs))`

eller `init xs = take (length xs - 1) xs`

Eksempel - quicksort

qs [] = []

? hent mindre/større (enn pivot) elementer fra listen:

mind x ls = for i in [0..length(ls)-1] ... ~~mind x ls = for i in~~
~~[0..length(ls)-1] ...~~

(mi) mind x [] = []

mind x (y:ys) = if (x > y) then (y:mind x ys) else (mind x ys)

(st) str x [] = []

str x (y:ys) = if (x <= y) then y:str x ys else str x ys

A. qs (x:xs) = mind x xs ++ [x] ++ str x xs **qs** (mind x
xs) ++ [x] ++ **qs** (str x xs)

(m) mind x **ys** = [y | y <- ys, y < x]

(s) str x ys = [y | y <- ys, y >= x]

qs (x:xs) = qs [y|y <- xs, y < x] ++ [x] ++ qs [y|y <- xs, y >= x]

? endre den til sortering minkende: rq [] = [] / rr [] = []

rq (x:xs) = rq [y|y <- xs, y >= x] ++ [x] ++ rq [y|y <- xs, y < x]

rr (x:xs) = rr [y|y <- xs, y > x] ++ [x] ++ rr [y|y <- xs, y <= x]